

Temporal Difference Learning

Presented by : **Veerav Chebrolu**
MSc. Scientific Computing

Date : **4 June 2024**

Motivation

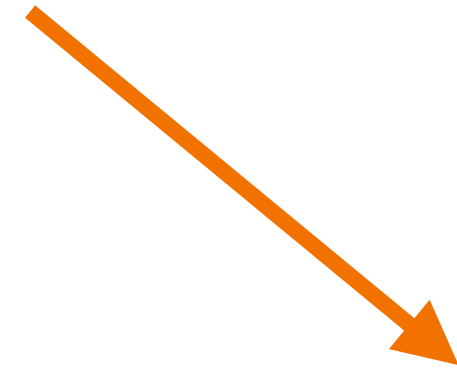
If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be temporal-difference (TD) learning.

Problem breakdown



Prediction

Evaluate the value function of a given a policy



Control

Optimise the value function



Unknown MDP

Recall : Monte Carlo Policy Evaluation

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

First-visit MC prediction, for estimating $V \approx v_{\pi}$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Sampling

Efficient Computation
of Returns

First visit

Recall : Incremental Mean

$$NewEstimate \leftarrow OldEstimate + StepSize * [Target - OldEstimate]$$

α

Error Estimate

$$\mu_k = \frac{1}{k} \sum_{j=1}^k x_j$$

$$= \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right)$$

$$= \frac{1}{k} (x_k + (k-1)\mu_{k-1})$$

$$= \mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1})$$

Advantages : Incremental Mean

$$NewEstimate \leftarrow OldEstimate + StepSize * [Target - OldEstimate]$$

$$\mu_k = \frac{1}{k} \sum_{j=1}^k x_j$$

$$= \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right)$$

$$= \frac{1}{k} (x_k + (k-1)\mu_{k-1})$$

α

Error Estimate

$$\alpha x_k + (1 - \alpha)\mu_{k-1}$$

Exponential Moving Average

$$\mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1})$$

Forget Old Episodes in Non Stationary Environment, by adjusting α

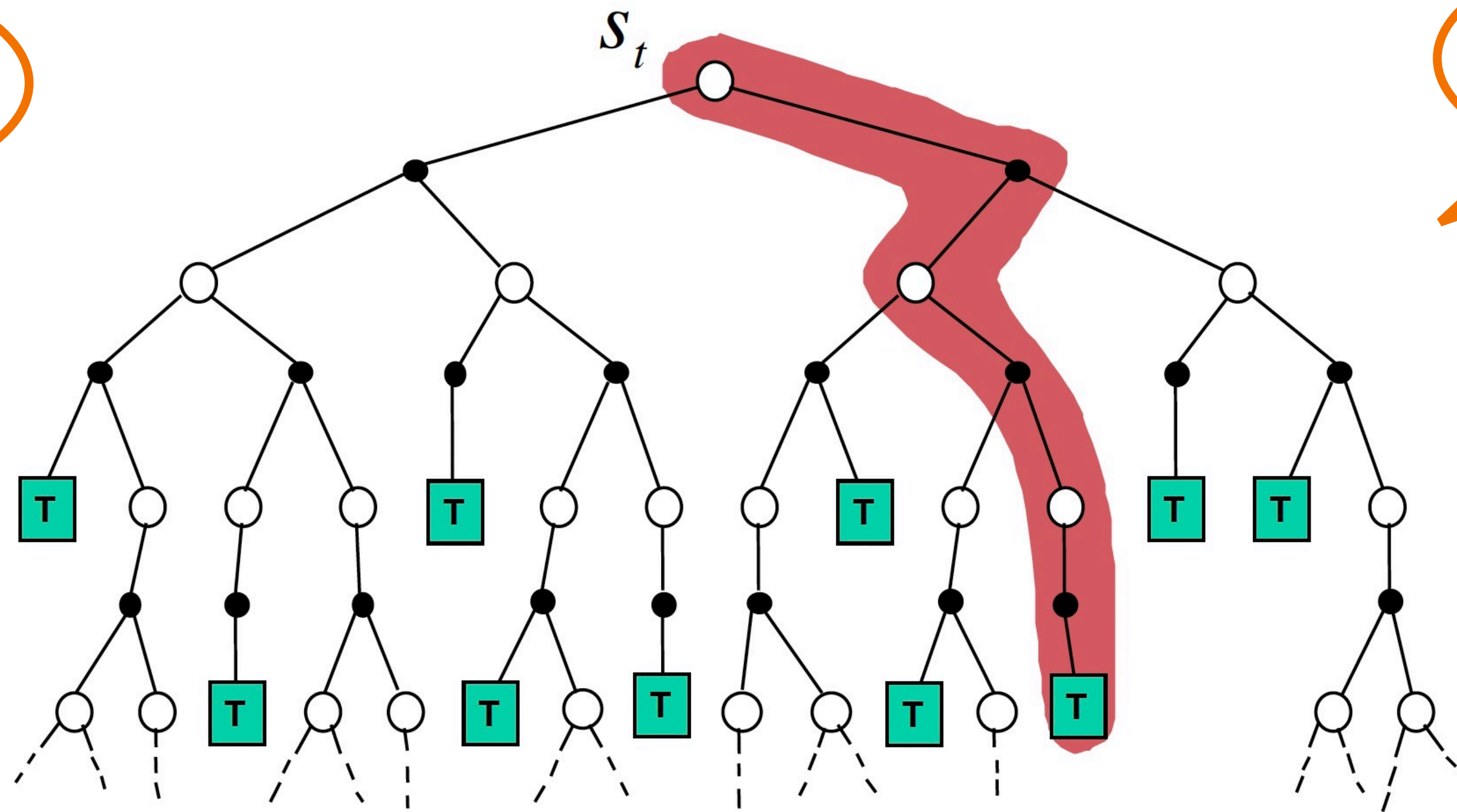
Recall : Monte Carlo Update

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

Actual Return

Model Free

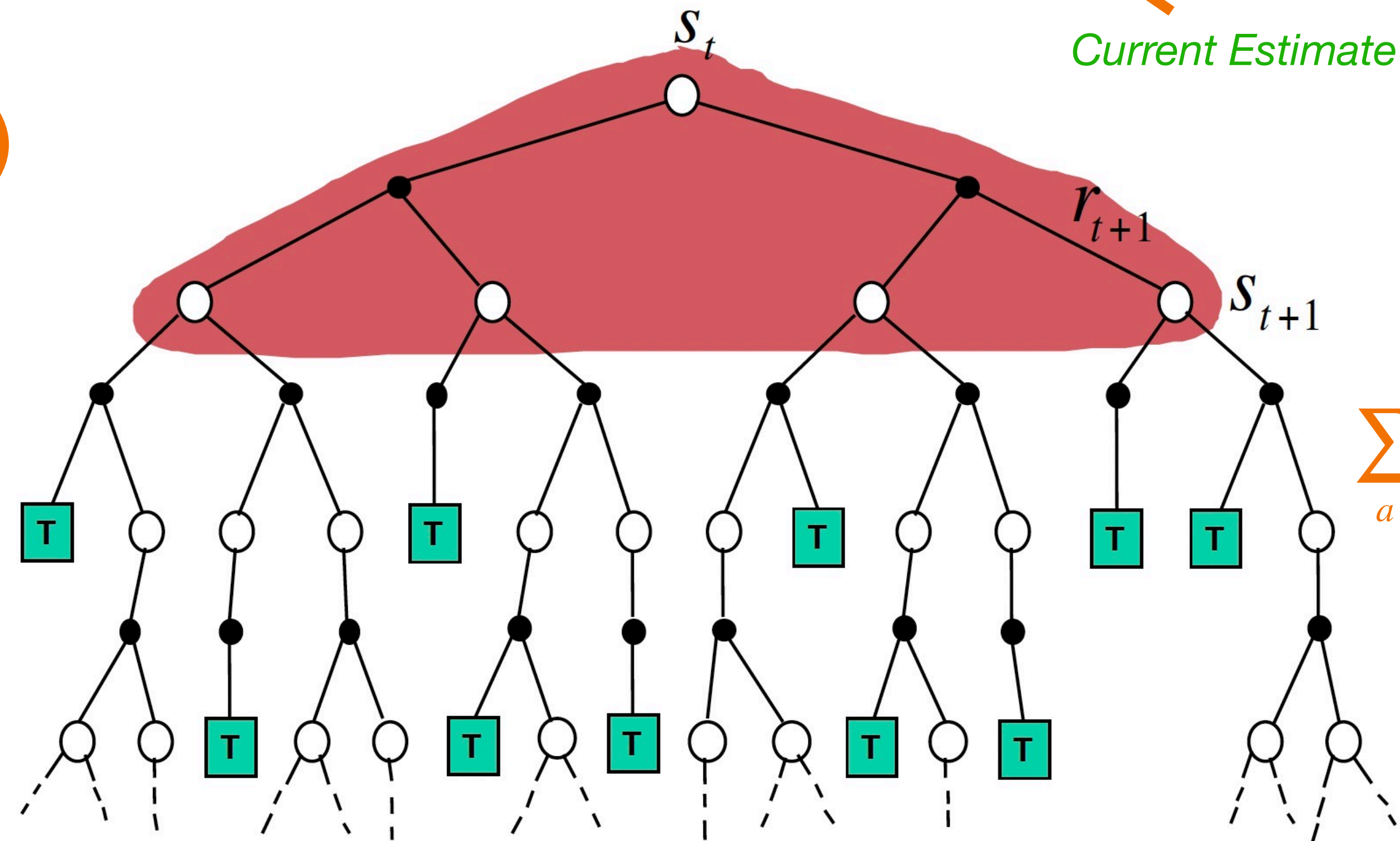
Sampling



Recall : Dynamic Programming Backup

$$V(S_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma V(S_{t+1})]$$

Model Based



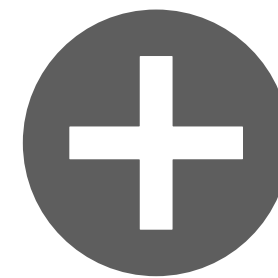
$$\sum_a \pi(a | s) \sum_{s', r} P(s', r | s, a) [r + \gamma V(s')]$$

TD Learning

Monte Carlo



Sampling



Dynamic Programming

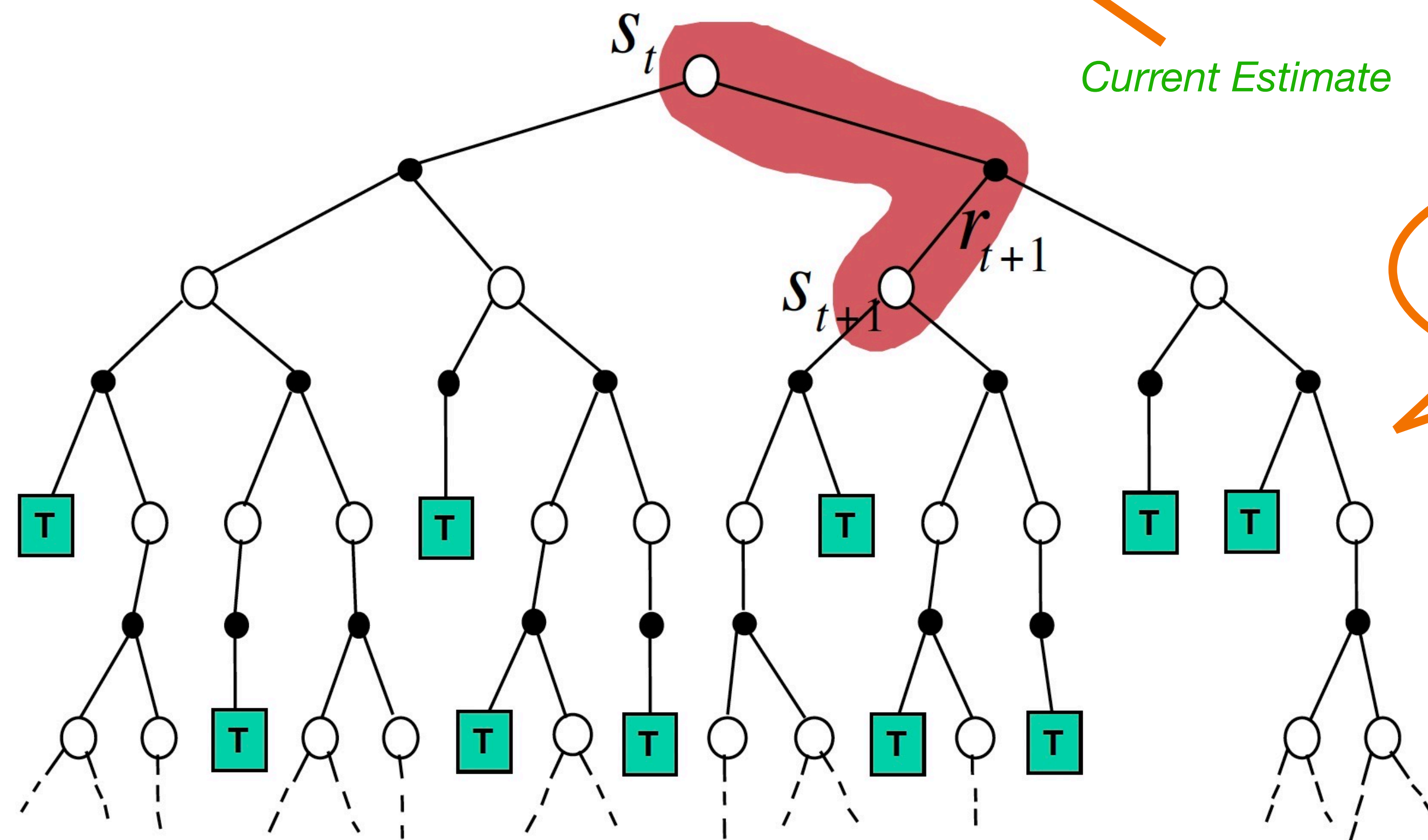


Bootstrapping

Update is based on
existing estimate

Temporal Difference (0) update

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



Model Free

Bootstrapping

TD Update

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{Target}}$$

TD Error

Target

TD(0) Policy Evaluation

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

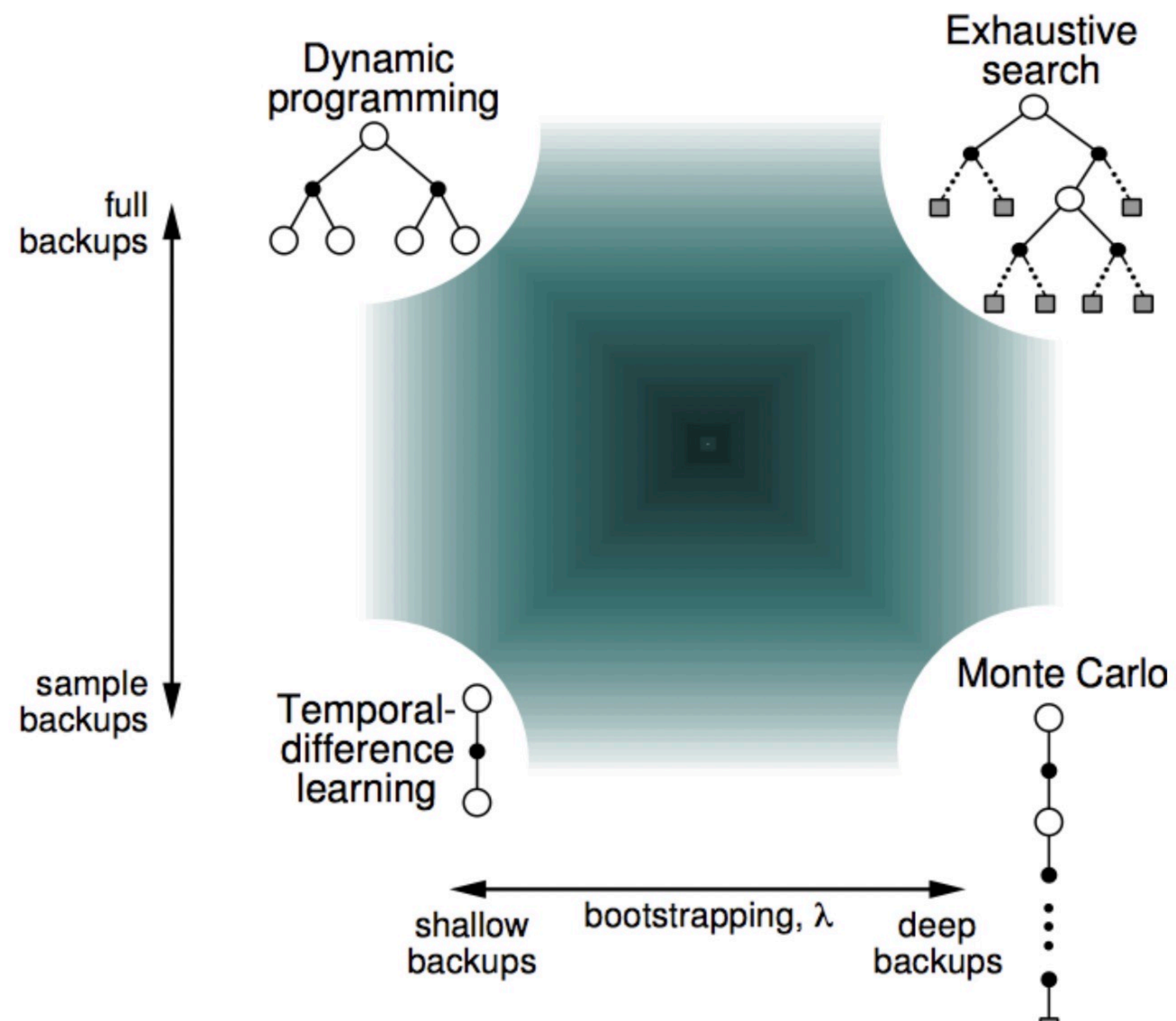
 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

Unified View Of Reinforcement Learning

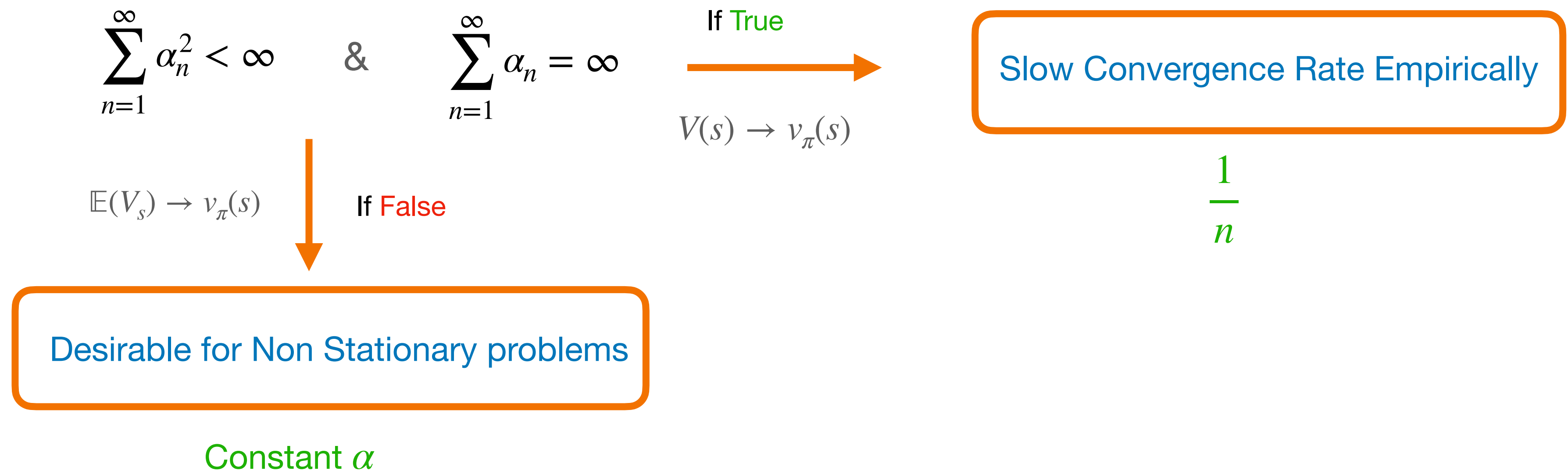


Advantages of TD Prediction

- TD can learn *before* knowing the final outcome
 - TD can learn online after every step
 - MC must wait until end of episode before return is known
- TD can learn *without* the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments

Are **TD** methods sound ?

For any fixed policy, **TD(0)** has been proved to converge to v_π , in the mean for a constant α if it is sufficiently small, and with probability 1 if α decreases as

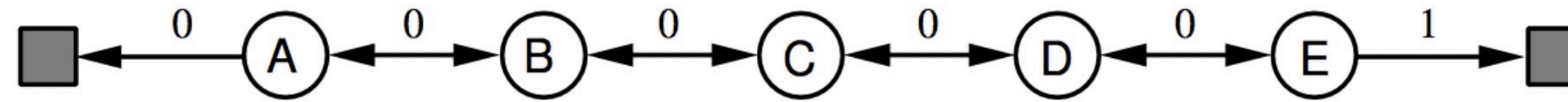


TD vs Monte Carlo : Which method **learns faster ?**

This is an Open Question

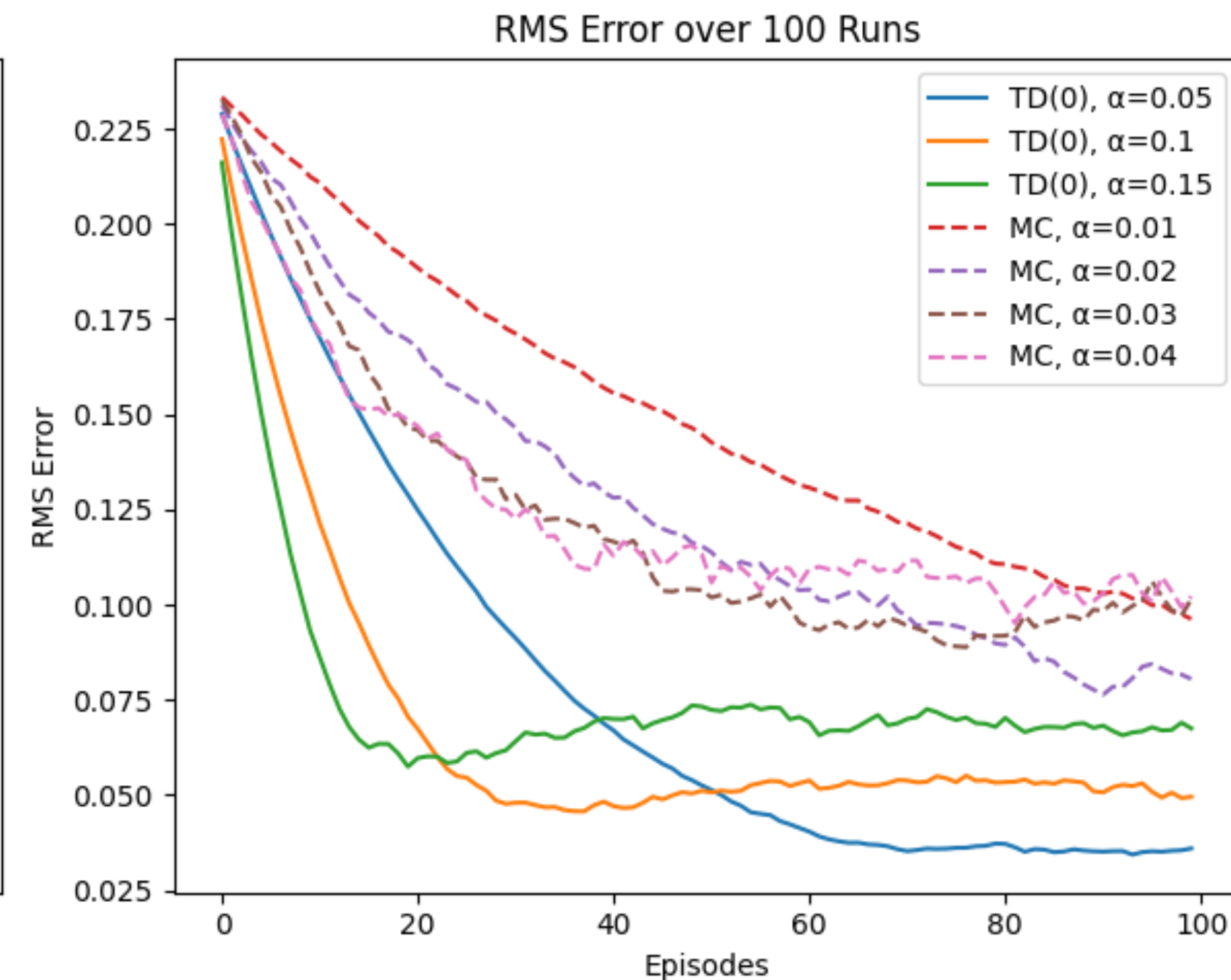
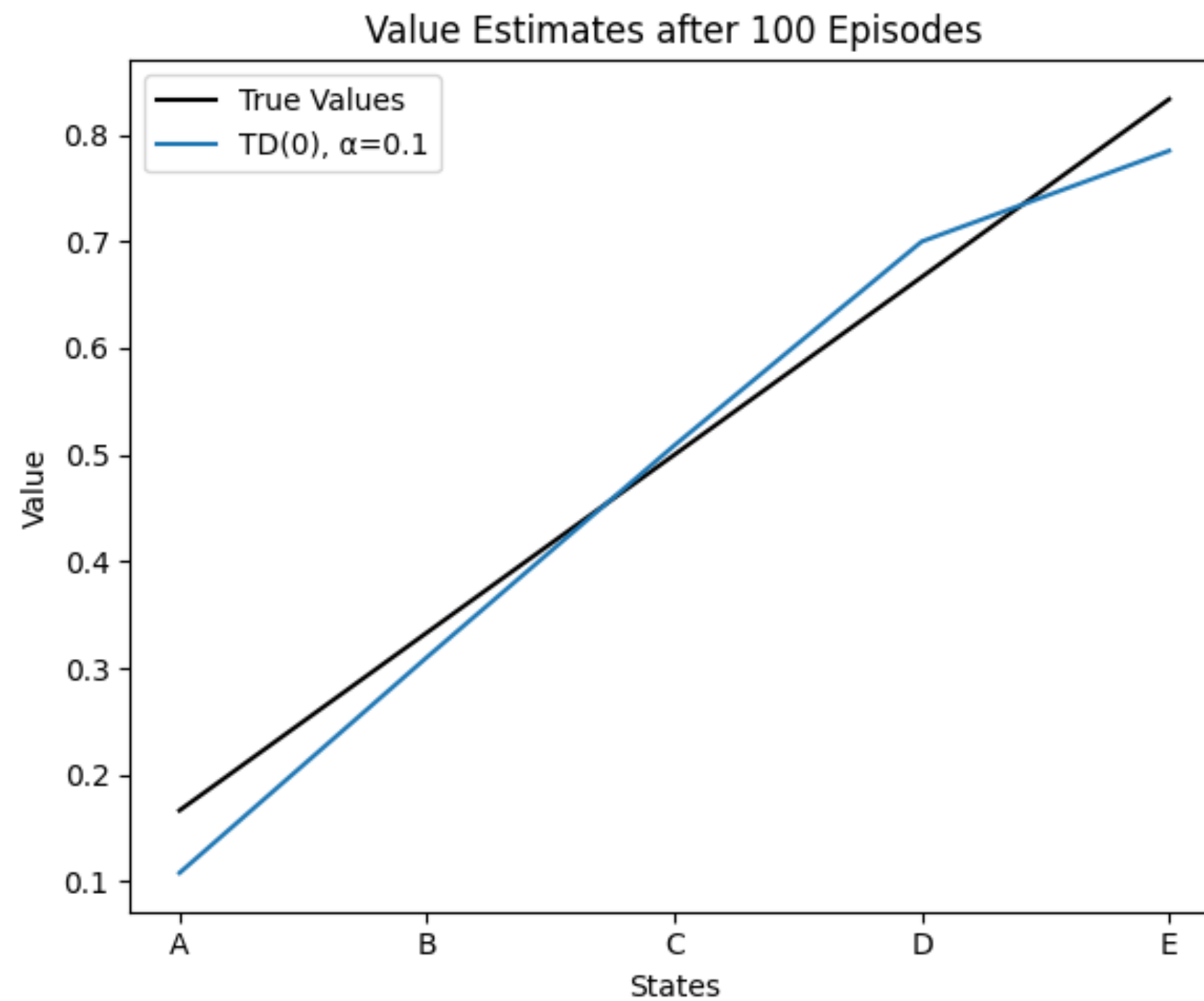
Empirical Evidences

Example 6.2 Random Walk



Start

```
true_values = {'A': 1/6, 'B': 2/6, 'C': 3/6, 'D': 4/6, 'E': 5/6}
```



Batch MC and TD (To show **Optimality**)

- MC and TD converge: $V(s) \rightarrow v_\pi(s)$ as experience $\rightarrow \infty$
- But what about batch solution for finite experience?

$$s_1^1, a_1^1, r_2^1, \dots, s_{T_1}^1$$

$\vdots$

$$s_1^K, a_1^K, r_2^K, \dots, s_{T_K}^K$$

Compute TD error
after each step

- e.g. Repeatedly sample episode $k \in [1, K]$
- Apply MC or TD(0) to episode k

Update value function
after entire batch,
with the sum of errors

Compute MC error
after each episode

AB example

Two states A, B ; no discounting; 8 episodes of experience

$A, 0, B, 0$

$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 0$

What is $V(A), V(B)$?

AB example

Two states A , B ; no discounting; 8 episodes of experience

$A, 0, B, 0$

$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 0$

What is $V(A)$, $V(B)$?

MC

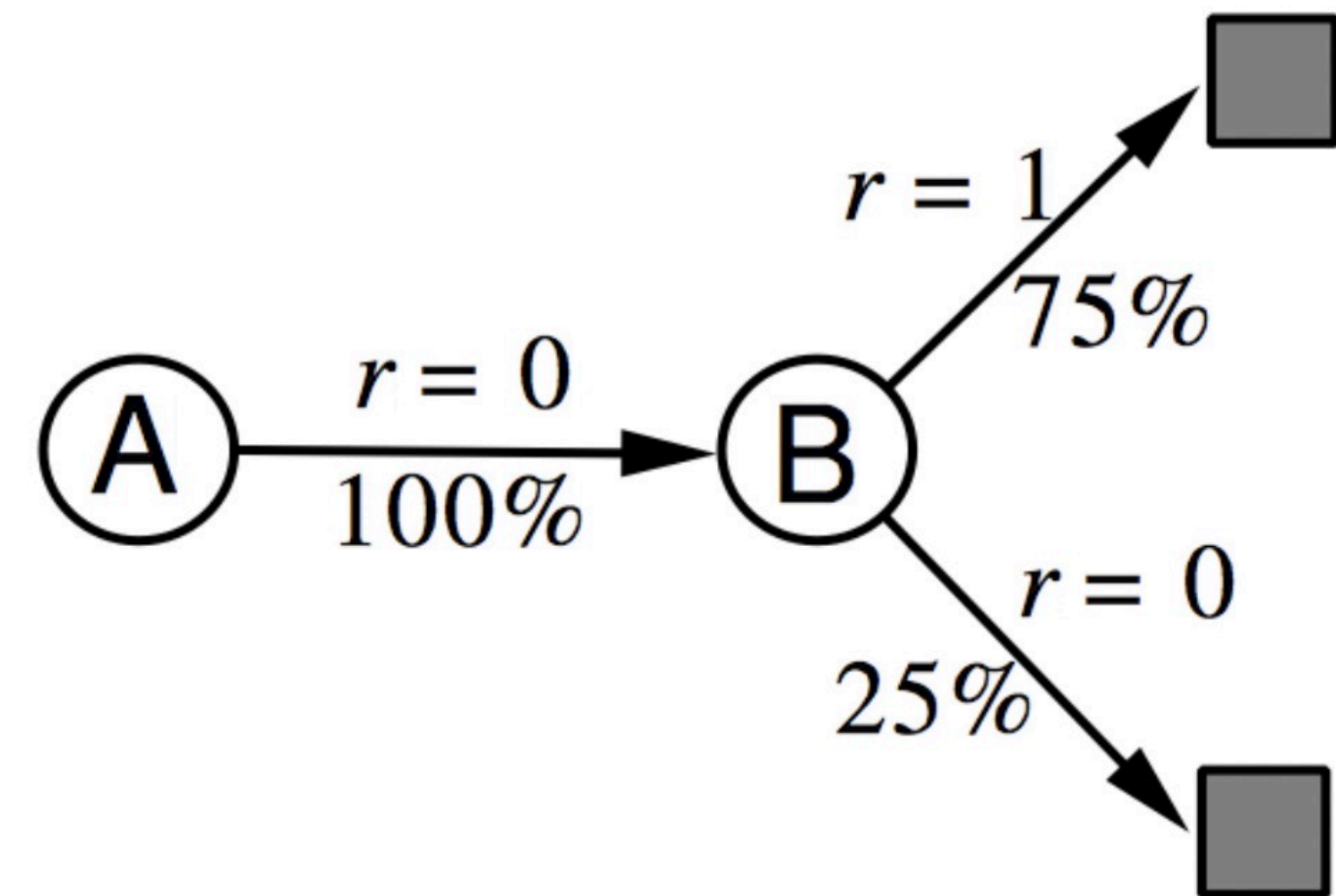


$V(A)=0$

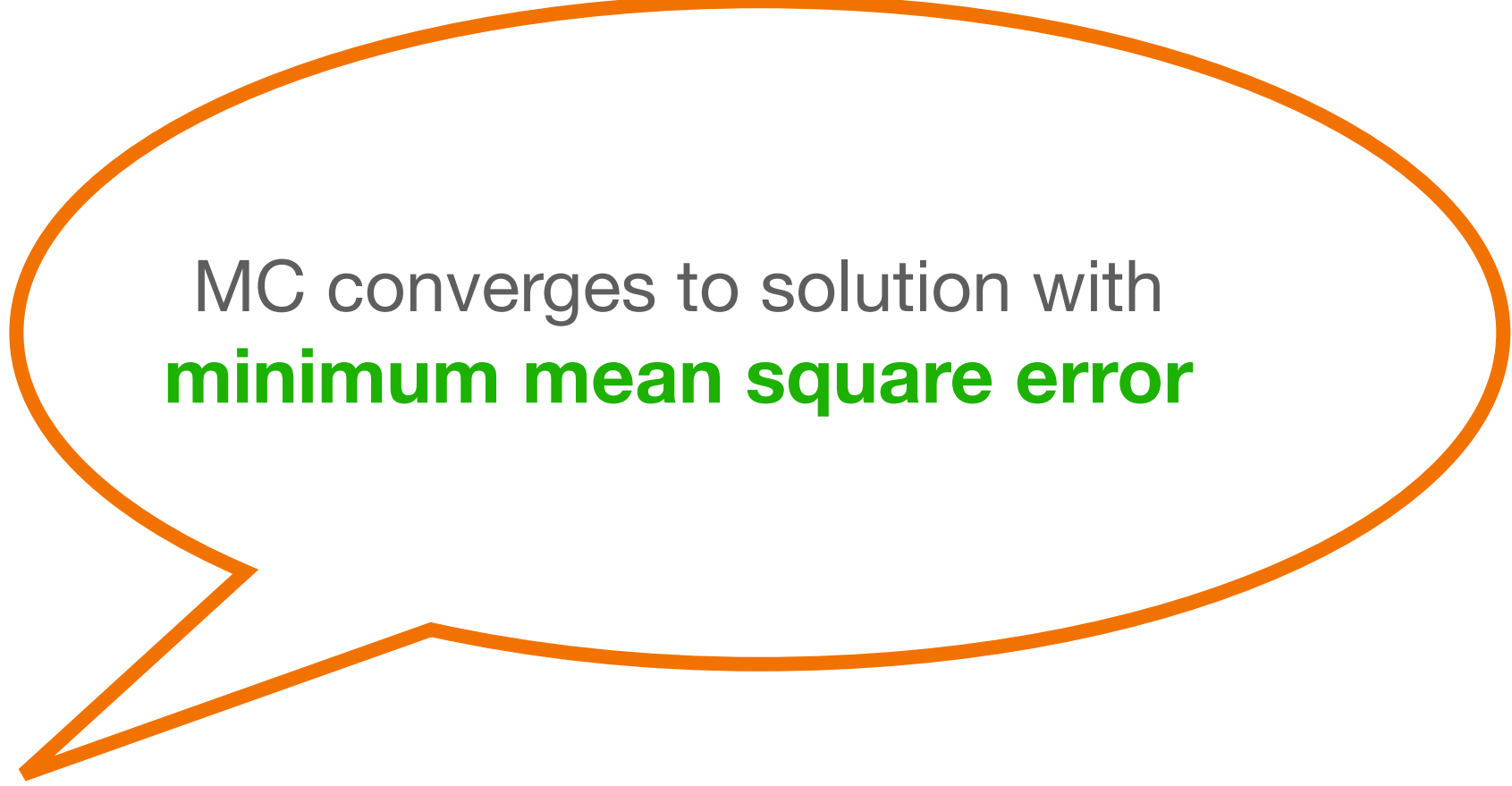
TD



$V(A)=3/4$



Certainty Equivalence



MC converges to solution with
minimum mean square error



TD converges to solution with
Maximum likelihood Markov Mode

TD Exploits Markov Property

TD CONTROL

Optimize the value function of an unknown MDP

On Policy



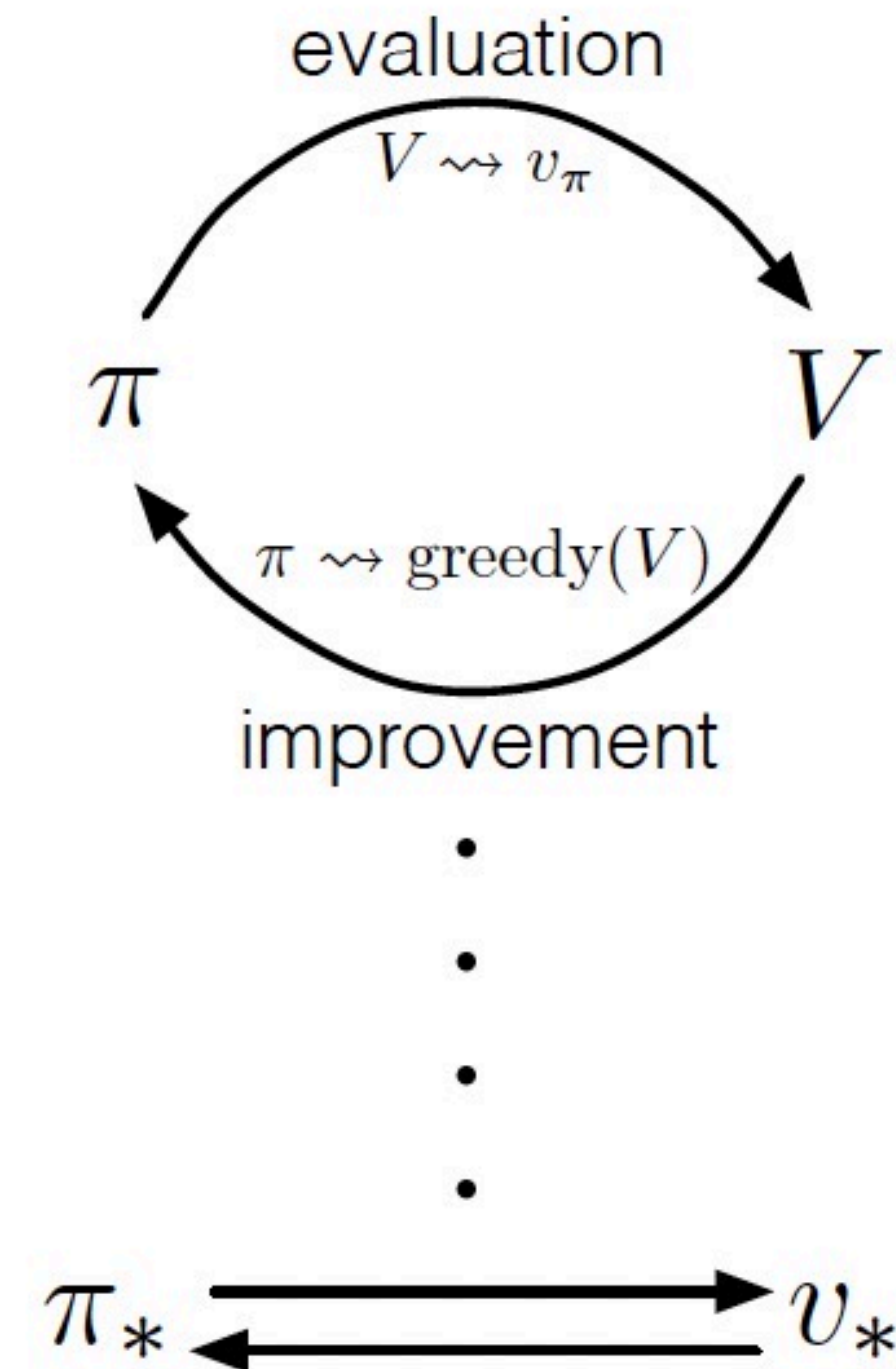
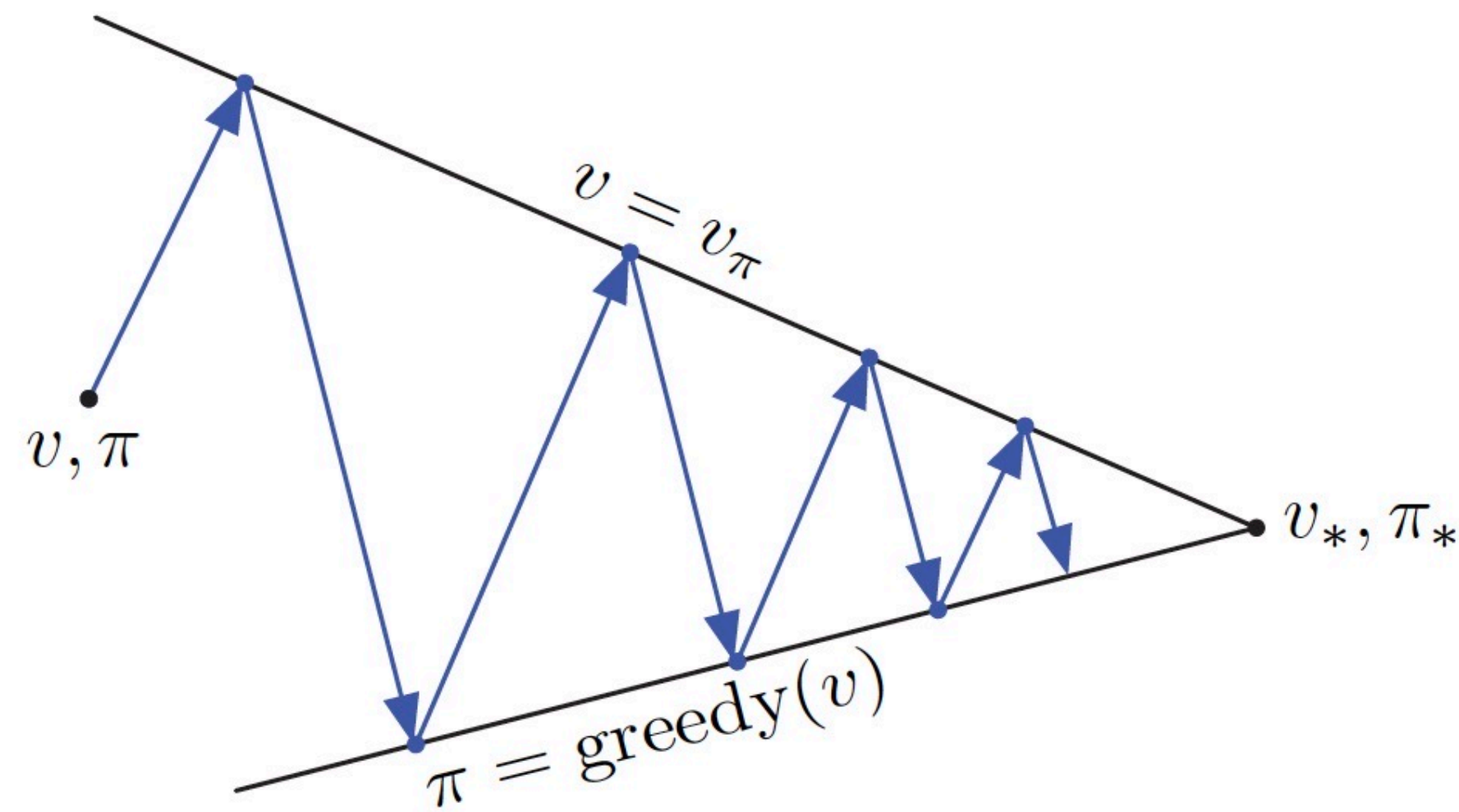
Learn about policy π from experience sampled from π

Off Policy



Learn about policy π from experience sampled from μ

Recall : Generalised Policy Iteration using iterative policy evaluation



Recall : Model Free Policy Iteration using Action Value Functions

- Greedy policy improvement over $V(s)$ requires model of MDP

$$\pi'(s) = \operatorname{argmax}_{a \in \mathcal{A}} \mathcal{R}_s^a + \mathcal{P}_{ss'}^a V(s')$$

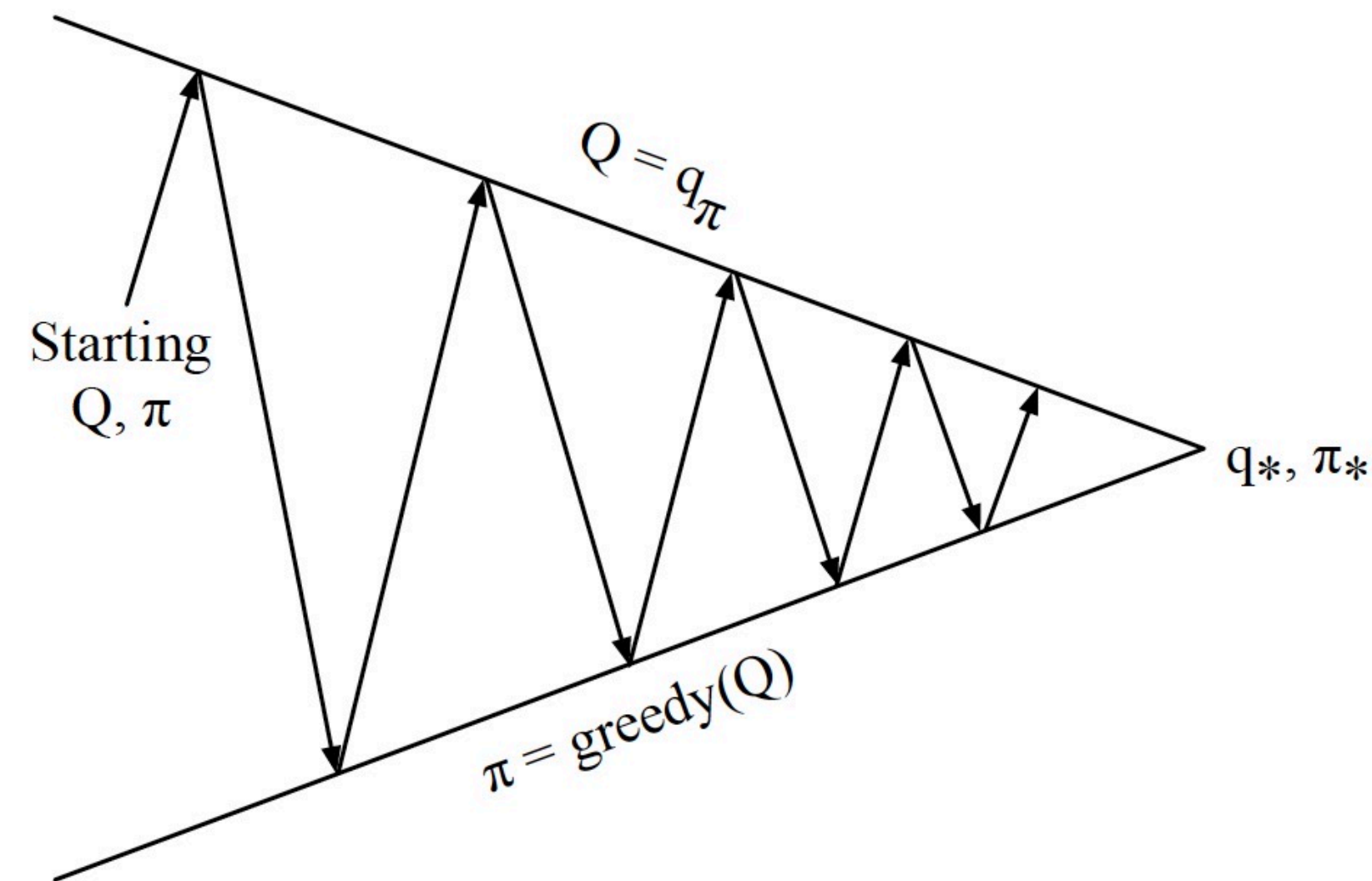
Requires Model

- Greedy policy improvement over $Q(s, a)$ is model-free

$$\pi'(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a)$$

Use Q value

Recall: Generalised Policy Iteration with Action Value function



Policy evaluation Monte-Carlo policy evaluation, $Q = q_\pi$

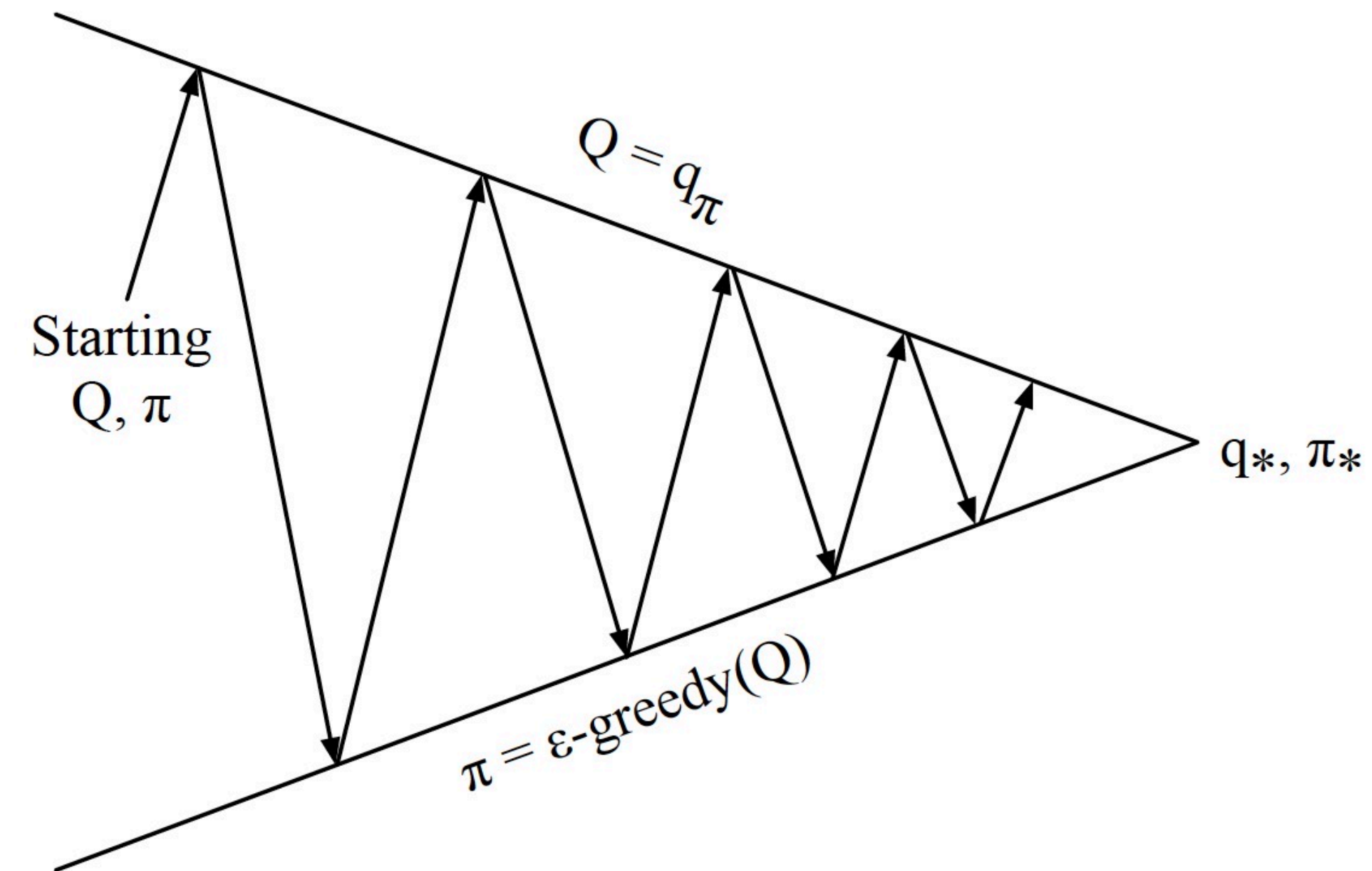
Policy improvement Greedy policy improvement?

Recall : ϵ greedy policy improvement

Theorem

For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to q_π is an improvement, $v_{\pi'}(s) \geq v_\pi(s)$

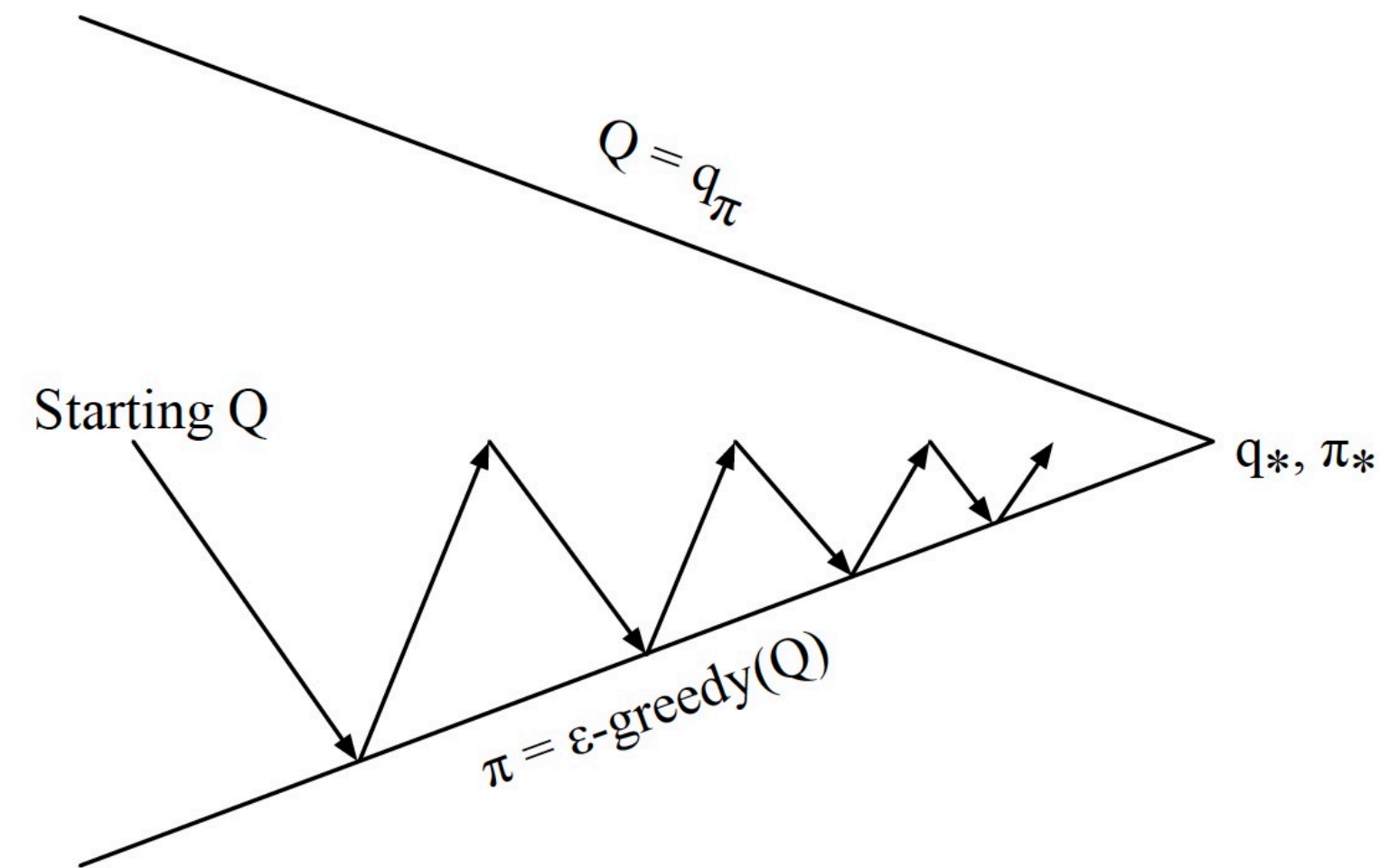
Recall : Monte Carlo Policy Iteration



Policy evaluation Monte-Carlo policy evaluation, $Q = q_\pi$

Policy improvement ϵ -greedy policy improvement

Recall : Monte Carlo Control



Every Episode :

Policy Evaluation : Monte Carlo, $Q \approx q_\pi$

Policy Improvement : $\sim \epsilon$ greedy

The Sarsa Algorithm

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

S_t

A_t

R_{t+1}

S_{t+1}

A_{t+1}

Evaluation

$\sim \pi$



+0



The Sarsa Algorithm

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

S_t

A_t

R_{t+1}

S_{t+1}

A_{t+1}

Improvement

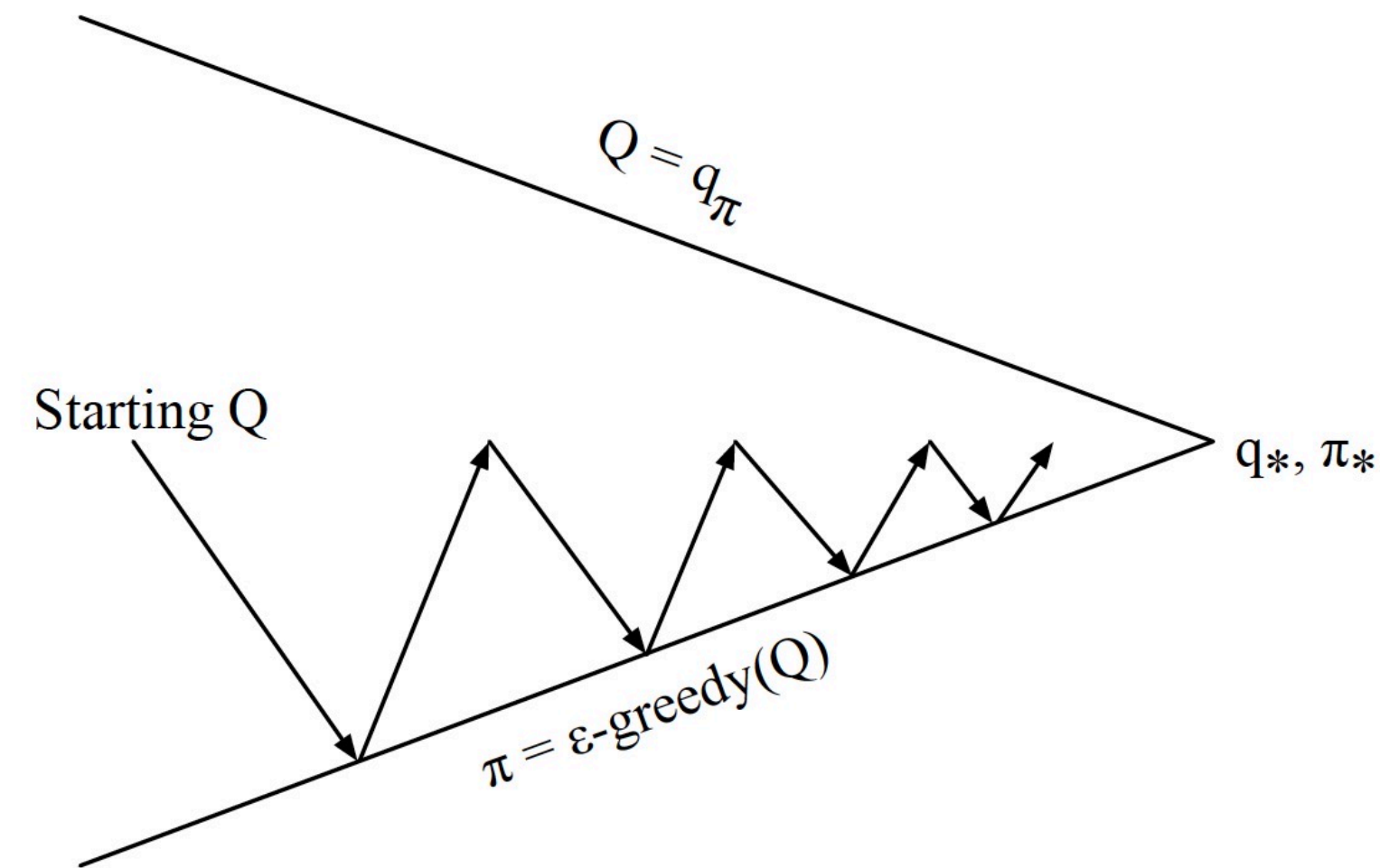
$\sim \epsilon$ greedy



+0



On Policy Control with Sarsa



Every **Time Step** :

Policy Evaluation : **Sarsa**, $Q \approx q_\pi$

Policy Improvement : $\sim \epsilon$ greedy

The Sarsa Algorithm

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

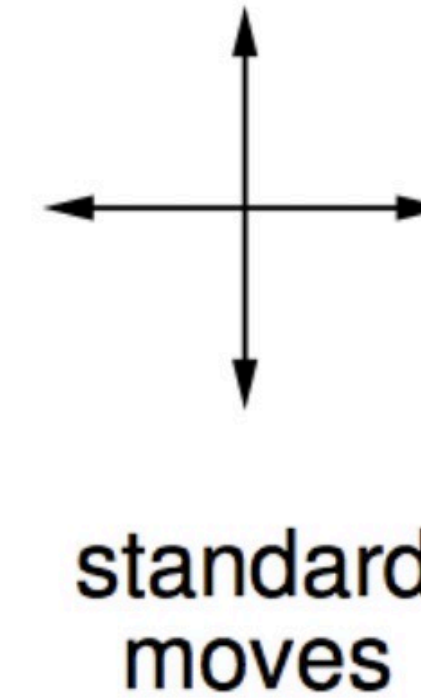
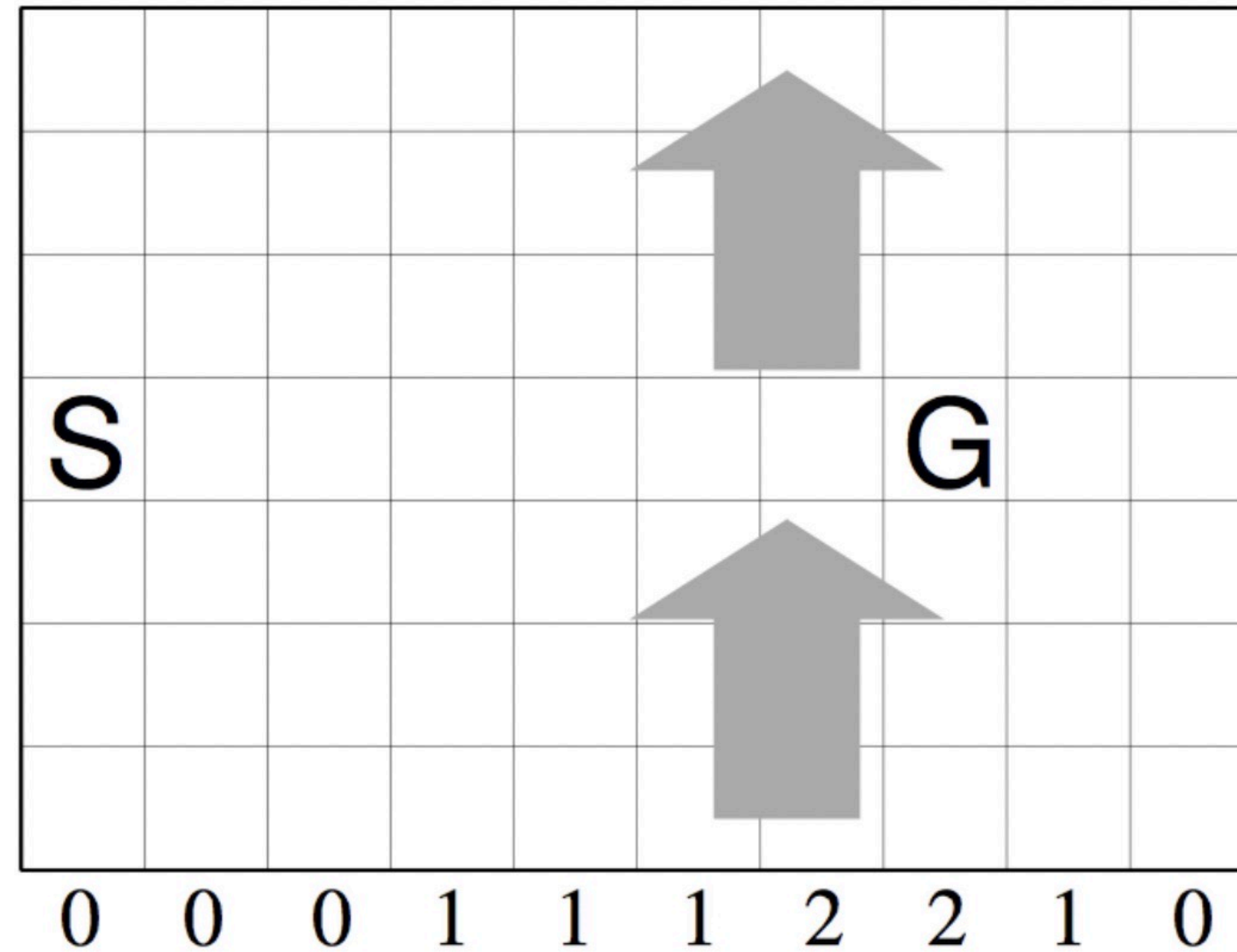
 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

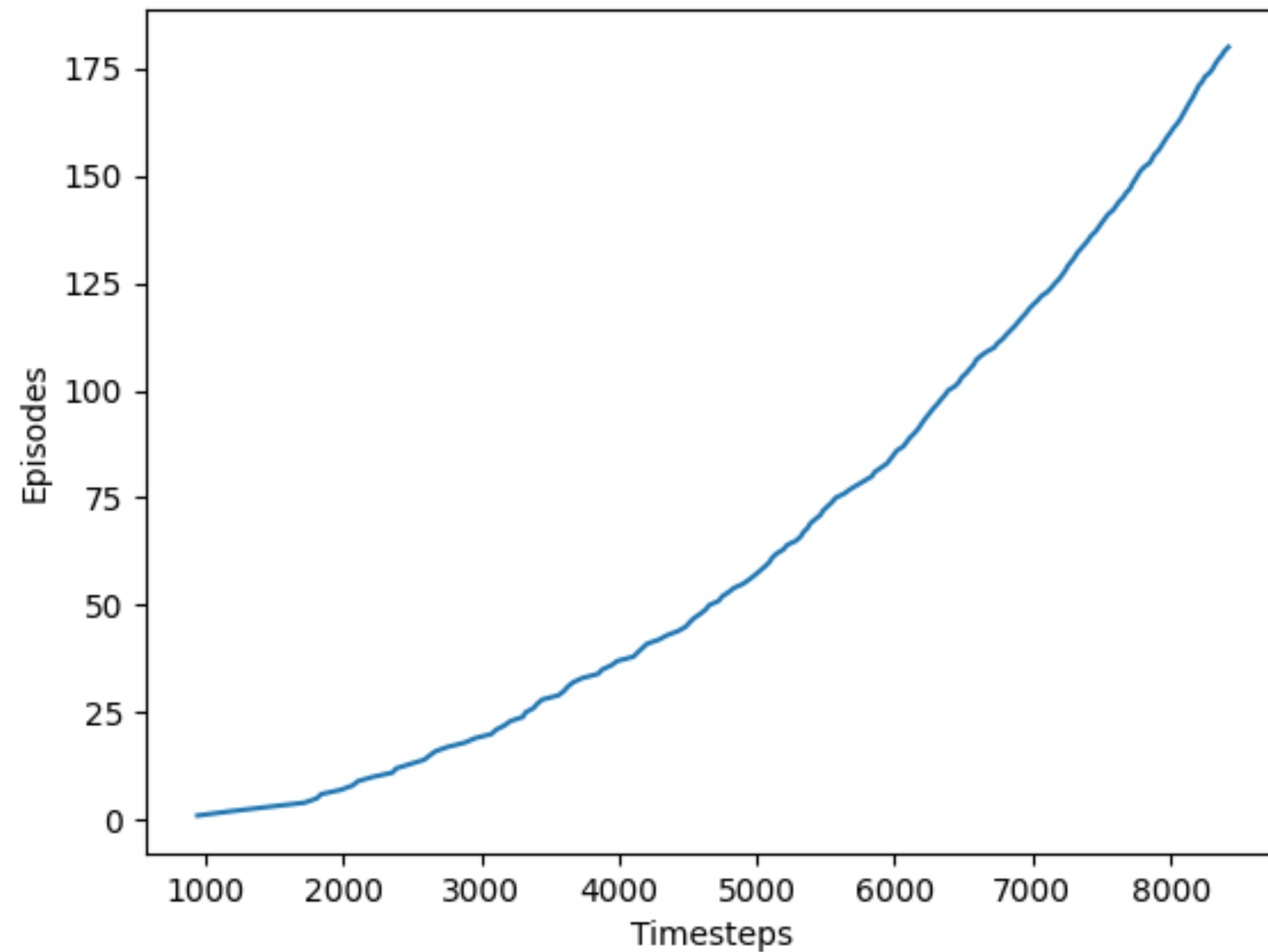
Windy Grid World



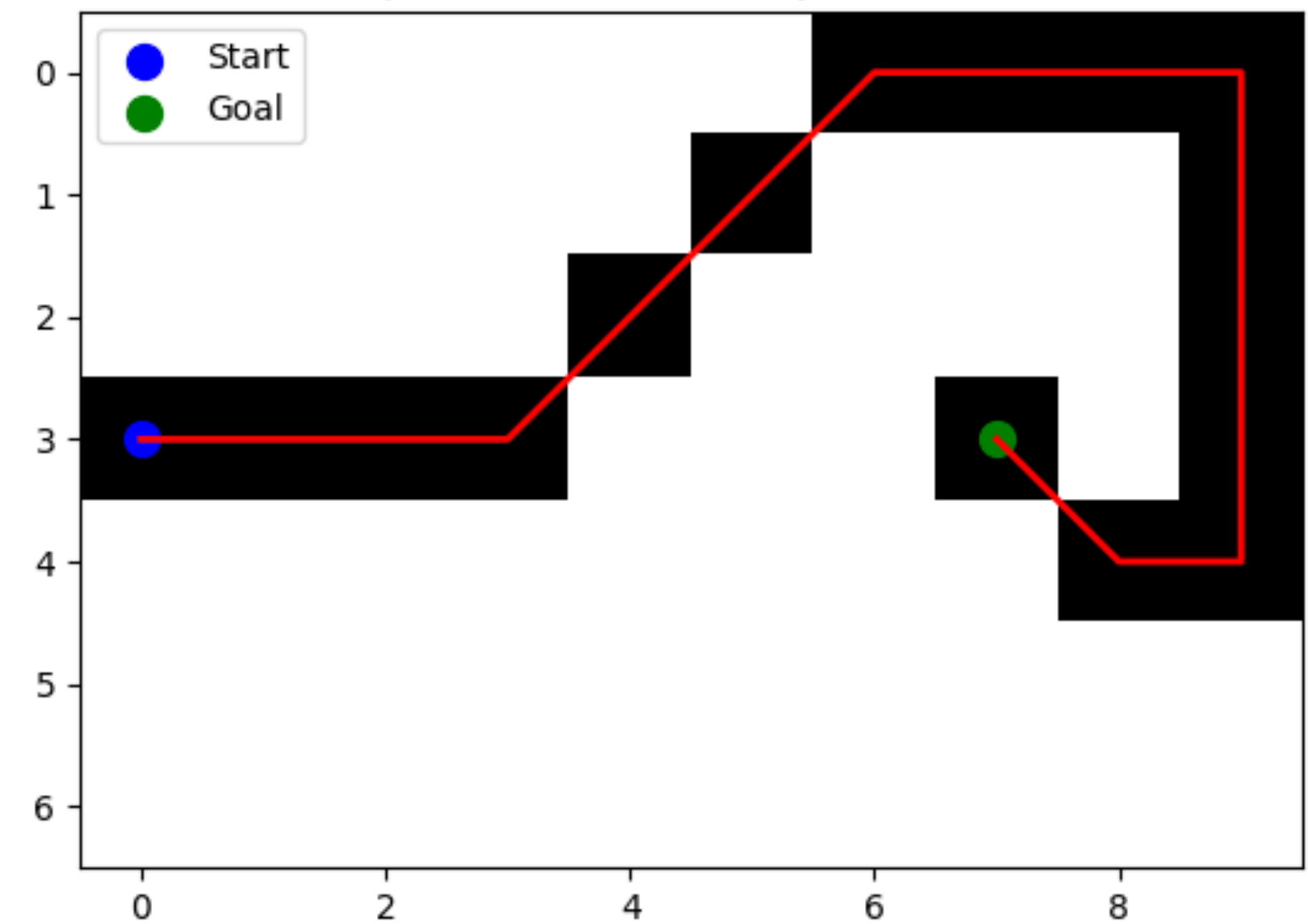
- Reward = -1 per time-step until reaching goal
- Undiscounted

Windy Grid World

Windy Gridworld: Episodes vs. Timesteps using Sarsa(0)



Optimal Path on Windy Gridworld



Q Learning : Off Policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$$S \leftarrow S'$$

 until S is terminal

SARSA

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Repeat (for each step of episode):
 Take action A , observe R, S'
 Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)
 $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$
 $S \leftarrow S'; A \leftarrow A'$
 until S is terminal

Figure 6.9: Sarsa: An on-policy TD control algorithm.

Q learning

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$; **what about A' ?**
 until S is terminal

TLDR;

	SARSA	Q-learning
Choosing A'	π	π
Updating Q	π	μ

Cliff Environment

